

¿Qué es t-SNE?

Objetivo

El algoritmo **t-SNE (t-Distributed Stochastic Neighbor Embedding)** es una técnica de **reducción de dimensionalidad no lineal** diseñada para visualizar conjuntos de datos de alta dimensión en un espacio de dos o tres dimensiones.

Su objetivo principal es preservar las relaciones de vecindad entre las observaciones.

Es decir:

- Si dos observaciones son muy similares en el espacio original, t-SNE intentará colocarlas cerca una de otra en el espacio reducido.
 - Si dos observaciones son muy diferentes, t-SNE tenderá a separarlas.
-

Aplicaciones

- Visualización de datos de alta dimensión.
 - Exploración de datasets.
 - Agrupamiento visual.
 - Análisis de embeddings.
 - Redes neuronales profundas.
 - Procesamiento de lenguaje natural.
 - Visión por computador.
 - Bioinformática.
-

Ejemplo

Suponga un conjunto de datos formado por vectores de 100 dimensiones.

$$\mathbf{x}_1 = \begin{bmatrix} 2 \\ 5 \\ \vdots \\ 8 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 3 \\ 4 \\ \vdots \\ 9 \end{bmatrix}$$

Cada observación posee 100 características.

Resulta imposible visualizar directamente estos datos.

t-SNE construye una representación equivalente en únicamente dos dimensiones.

$$\mathbf{y}_1 = \begin{bmatrix} 1.8 \\ 2.1 \end{bmatrix}, \quad \mathbf{y}_2 = \begin{bmatrix} 2.0 \\ 2.3 \end{bmatrix}$$

Ahora es posible representar todas las observaciones en un plano cartesiano.

¿Por qué reducir la dimensionalidad?

Muchos algoritmos de Machine Learning trabajan con cientos o miles de variables.

Ejemplos:

Problema	Número de características
Iris	4
MNIST	784
CIFAR-10	3072
Embeddings BERT	768
Embeddings GPT	1536 o más

Problema

Los seres humanos únicamente podemos visualizar:

- una dimensión
- dos dimensiones
- tres dimensiones

No podemos imaginar espacios de cientos de dimensiones.

Objetivo

Encontrar una representación

$$\mathbb{R}^D \longrightarrow \mathbb{R}^2$$

o

$$\mathbb{R}^D \longrightarrow \mathbb{R}^3$$

que conserve la mayor cantidad posible de información.

Idea

En lugar de visualizar

$$(3.2, 5.4, 1.7, 9.1, 4.8, \dots)$$

preferimos obtener

$$(2.14, 1.87)$$

de manera que los puntos cercanos permanezcan cercanos.

Ese es precisamente el objetivo de t-SNE.

Comparación entre PCA y t-SNE

Existen numerosas técnicas de reducción de dimensionalidad.

Las más conocidas son:

- PCA
 - t-SNE
 - UMAP
-

PCA

PCA busca nuevas variables mediante combinaciones lineales.

$$\mathbf{Y} = \mathbf{XW}$$

donde

- $\mathbf{X}$ es la matriz de datos.
- $\mathbf{W}$ contiene los autovectores.

PCA intenta conservar la mayor varianza posible.

t-SNE

t-SNE no busca conservar la varianza.

Su objetivo consiste en preservar la estructura de vecindad.

Dos puntos próximos continúan próximos.

Dos puntos muy alejados permanecen separados.

Comparación

PCA	t-SNE
Lineal	No lineal
Muy rápido	Más lento
Conserva la varianza	Conserva vecindades
Fácil de interpretar	Difícil de interpretar
Bueno para compresión	Excelente para visualización

Ejemplo

Suponga tres puntos

A, B y C

Si

$$d(A, B) = 1$$

y

$$d(A, C) = 10$$

t-SNE intentará mantener aproximadamente esa relación en el espacio reducido.

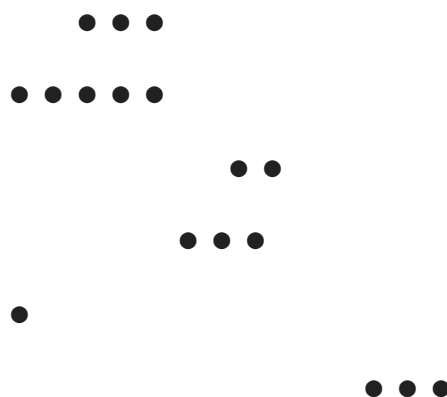
No necesariamente conservará las distancias exactas, sino las relaciones de vecindad.

Intuición del algoritmo t-SNE

Imagine una hoja de papel con miles de puntos dibujados.

Cada punto representa una observación.

Los puntos similares forman pequeños grupos.



Ahora imagine que esa hoja se encuentra doblada muchas veces.

Los puntos pasan a ocupar un espacio de cientos de dimensiones.

Nuestro objetivo consiste en desplegar nuevamente la hoja sin romper los grupos.

Idea fundamental

t-SNE calcula la probabilidad de que dos observaciones sean vecinas.

Si dos observaciones son muy parecidas,

$$P_{ij} \approx 1$$

Si son muy diferentes,

$$P_{ij} \approx 0$$

Después construye nuevas coordenadas

$$\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$$

de forma que las probabilidades calculadas en dos dimensiones sean lo más parecidas posible.

En resumen,

1. Calcula distancias entre todos los puntos.
2. Convierte esas distancias en probabilidades.
3. Coloca los puntos en dos dimensiones.
4. Mueve continuamente los puntos para minimizar el error entre ambas distribuciones.

En las siguientes diapositivas comenzaremos estudiando el primer paso del algoritmo:

el cálculo de la matriz de distancias (`pdist`).

Distancia Euclidiana

La primera etapa del algoritmo **t-SNE** consiste en medir qué tan parecidas son las observaciones del conjunto de datos.

Para ello se calcula la **distancia euclidiana** entre todos los pares de observaciones.

Si dos puntos están muy cerca, se consideran similares.

Si están muy separados, se consideran diferentes.

Distancia Euclidiana

Sea

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix}$$

y

$$\mathbf{x}_j = \begin{bmatrix} x_{j1} \\ x_{j2} \\ \vdots \\ x_{jd} \end{bmatrix}$$

Entonces

$$d_{ij} = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

Interpretación

- Distancia pequeña → puntos similares.
- Distancia grande → puntos diferentes.

Toda la información utilizada posteriormente por t-SNE comienza con estas distancias.

Derivación de la distancia euclidiana

Consideremos dos puntos en dos dimensiones

$$A(x_1, y_1)$$

y

$$B(x_2, y_2)$$

La diferencia horizontal es

$$\Delta x = x_2 - x_1$$

La diferencia vertical es

$$\Delta y = y_2 - y_1$$

Aplicando el teorema de Pitágoras,

$$d^2 = (\Delta x)^2 + (\Delta y)^2$$

Despejando la distancia,

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

Generalizando a un espacio de D dimensiones,

$$d_{ij} = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots + (x_{id} - x_{jd})^2}$$

Observación

El algoritmo t-SNE utiliza esta fórmula para **cada par de observaciones** del conjunto de datos.

Cálculo de la distancia euclidiana entre dos puntos

Utilizaremos el siguiente conjunto de datos durante todo el curso.

Punto	x	y
A	1	1
B	2	2
C	5	4
D	6	5

Distancia entre A y B

$$A = (1, 1)$$

$$B = (2, 2)$$

Aplicando la fórmula,

$$\begin{aligned} d(A, B) &= \sqrt{(2 - 1)^2 + (2 - 1)^2} \\ &= \sqrt{1^2 + 1^2} \end{aligned}$$

$$= \sqrt{2}$$

$$= 1.4142$$

Distancia entre A y C

$$d(A, C) = \sqrt{(5 - 1)^2 + (4 - 1)^2}$$

$$= \sqrt{4^2 + 3^2}$$

$$= \sqrt{25}$$

$$= 5$$

Interpretación

A está mucho más cerca de B que de C.

Por lo tanto,

- A y B deberían permanecer cercanos en el mapa bidimensional.
- A y C deberían aparecer bastante separados.

¿Qué calcula `pdist()`?

En t-SNE necesitamos conocer la distancia entre **todos los pares de observaciones**.

Consideremos nuevamente nuestro conjunto de datos.

Punto	x	y
A	1	1
B	2	2
C	5	4
D	6	5

Cada punto representa una observación del conjunto de datos.

¿Cuántas distancias existen?

Con cuatro observaciones existen

$$\frac{n(n-1)}{2}$$

distancias diferentes.

Sustituyendo

$$n = 4$$

obtenemos

$$\frac{4(4-1)}{2} = 6$$

Por lo tanto, debemos calcular únicamente las siguientes distancias:

Distancia

$$d(A, B)$$

$$d(A, C)$$

$$d(A, D)$$

$$d(B, C)$$

$$d(B, D)$$

$$d(C, D)$$

que se traduce en la construcción del siguiente vector:

$$[d_{AB}, d_{AC}, d_{AD}, d_{BC}, d_{BD}, d_{CD}]$$

¿Por qué solamente seis?

La distancia entre dos puntos es simétrica.

$$d(A, B) = d(B, A)$$

Además,

$$d(A, A) = 0$$

Por esta razón no es necesario almacenar la matriz completa.

Basta guardar únicamente las distancias del triángulo superior de la matriz.

Este formato recibe el nombre de **vector condensado** (*condensed distance matrix*). Ese formato permite expandirse a una matriz completa de distancias que utilizará t-SNE para calcular las probabilidades de vecindad.

Cálculo manual de las seis distancias

Recordemos la fórmula de la distancia euclidiana.

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

1. Distancia entre A y B

$$\begin{aligned} d(A, B) &= \sqrt{(2-1)^2 + (2-1)^2} \\ &= \sqrt{2} = 1.4142 \end{aligned}$$

2. Distancia entre A y C

$$\begin{aligned}d(A, C) &= \sqrt{(5 - 1)^2 + (4 - 1)^2} \\&= \sqrt{16 + 9} = 5.0000\end{aligned}$$

3. Distancia entre A y D

$$\begin{aligned}d(A, D) &= \sqrt{(6 - 1)^2 + (5 - 1)^2} \\&= \sqrt{25 + 16} = 6.4031\end{aligned}$$

4. Distancia entre B y C

$$\begin{aligned}d(B, C) &= \sqrt{(5 - 2)^2 + (4 - 2)^2} \\&= \sqrt{9 + 4} = 3.6056\end{aligned}$$

5. Distancia entre B y D

$$\begin{aligned}d(B, D) &= \sqrt{(6 - 2)^2 + (5 - 2)^2} \\&= \sqrt{16 + 9} = 5.0000\end{aligned}$$

6. Distancia entre C y D

$$\begin{aligned}d(C, D) &= \sqrt{(6 - 5)^2 + (5 - 4)^2} \\&= \sqrt{2} = 1.4142\end{aligned}$$

Construcción de la matriz de distancias

Una vez calculadas todas las distancias, podemos organizarlas en una matriz simétrica.

$$D = \begin{bmatrix} 0 & d_{AB} & d_{AC} & d_{AD} \\ d_{BA} & 0 & d_{BC} & d_{BD} \\ d_{CA} & d_{CB} & 0 & d_{CD} \\ d_{DA} & d_{DB} & d_{DC} & 0 \end{bmatrix}$$

Sustituyendo los valores obtenidos,

$$D = \begin{bmatrix} 0 & 1.4142 & 5.0000 & 6.4031 \\ 1.4142 & 0 & 3.6056 & 5.0000 \\ 5.0000 & 3.6056 & 0 & 1.4142 \\ 6.4031 & 5.0000 & 1.4142 & 0 \end{bmatrix}$$

Observaciones

La diagonal principal siempre contiene ceros.

$$d(A, A) = 0$$

La matriz es simétrica.

$$D = D^T$$

Por ejemplo,

$$d(A, C) = d(C, A) = 5$$

Por esta razón la mitad inferior de la matriz contiene información repetida.

Construcción del vector condensado

Dado que la matriz de distancias es simétrica, resulta innecesario almacenar ambas mitades.

Únicamente se conserva el triángulo superior.

En nuestro ejemplo,

	A	B	C	D
A	0	1.4142	5.0000	6.4031
B		0	3.6056	5.0000
C			0	1.4142
D				0

Leyendo el triángulo superior de izquierda a derecha y de arriba hacia abajo obtenemos

$$[d_{AB}, d_{AC}, d_{AD}, d_{BC}, d_{BD}, d_{CD}]$$

Sustituyendo los valores,

$$[1.4142 \quad 5.0000 \quad 6.4031 \quad 3.6056 \quad 5.0000 \quad 1.4142]$$

Este será el resultado esperado

En la siguiente sección estudiaremos cómo la implementación de **MXNet** obtiene exactamente este mismo vector utilizando únicamente operaciones tensoriales, sin utilizar ningún bucle explícito.

Eliminación de los bucles mediante Álgebra Lineal

Hasta ahora calculamos las distancias utilizando la definición clásica.

Para dos observaciones,

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Esta fórmula funciona perfectamente, pero presenta un inconveniente.

Para un conjunto de datos con N observaciones sería necesario ejecutar dos bucles:

```
para i = 1 ... N
```

```
    para j = i+1 ... N
```

```
        calcular distancia(i,j)
```

Cuando N es muy grande, estos bucles se convierten en el principal cuello de botella del algoritmo.

Una mejor idea

En lugar de calcular una distancia a la vez,

podemos calcular **todas las distancias simultáneamente** mediante operaciones matriciales.

La clave consiste en eliminar la raíz cuadrada temporalmente y trabajar con la distancia al cuadrado.

$$d_{ij}^2 = \|\mathbf{x}_i - \mathbf{x}_j\|^2$$

Esto permite transformar el problema en productos matriciales altamente optimizados.

En las siguientes diapositivas veremos cómo hacerlo.

Expansión algebraica de la distancia al cuadrado

Partimos de la norma euclidiana.

$$\|\mathbf{x}_i - \mathbf{x}_j\|^2$$

Recordemos que

$$\|\mathbf{a}\|^2 = \mathbf{a}^T \mathbf{a}$$

Por lo tanto,

$$\|\mathbf{x}_i - \mathbf{x}_j\|^2 = (\mathbf{x}_i - \mathbf{x}_j)^T (\mathbf{x}_i - \mathbf{x}_j)$$

Aplicando la propiedad distributiva, obtenemos

$$\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \mathbf{x}_i^T \mathbf{x}_i - 2\mathbf{x}_i^T \mathbf{x}_j + \mathbf{x}_j^T \mathbf{x}_j$$

Esta identidad es la base de toda la implementación vectorizada, ya que reemplaza el cálculo directo de cada distancia.

¿Por qué es importante?

Observemos los tres términos.

$$\mathbf{x}_i^T \mathbf{x}_i$$

es simplemente la suma de cuadrados del punto i .

$$\mathbf{x}_j^T \mathbf{x}_j$$

es la suma de cuadrados del punto j .

Y

$$\mathbf{x}_i^T \mathbf{x}_j$$

es un producto punto.

Todos ellos pueden calcularse utilizando operaciones matriciales.

Primer paso: calcular las sumas de cuadrados

Consideremos nuevamente nuestro conjunto de datos.

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 5 & 4 \\ 6 & 5 \end{bmatrix}$$

Cada fila representa una observación.

La primera operación del algoritmo es

```
my $X_sum_squares =  
  nd->sum($X * $X,  
          axis => 1,  
          keepdims => 1);
```

Primero calculamos

$$X \odot X$$

donde $\odot$ representa la multiplicación elemento a elemento.

$$= \begin{bmatrix} 1 & 1 \\ 4 & 4 \\ 25 & 16 \\ 36 & 25 \end{bmatrix}$$

Ahora sumamos cada fila.

Punto	Cálculo	Resultado
A	$1 + 1$	2
B	$4 + 4$	8
C	$25 + 16$	41
D	$36 + 25$	61

Por lo tanto,

$$X_{\text{sum_squares}} = \begin{bmatrix} 2 \\ 8 \\ 41 \\ 61 \end{bmatrix}$$

Cada elemento corresponde exactamente a

$$\mathbf{x}_i^T \mathbf{x}_i$$

de la identidad algebraica. Por lo tanto, la operación `nd->sum()` genera el vector de normas cuadradas de todas las observaciones.

Segundo paso: calcular todos los productos punto

La segunda operación es

```
my $X_inner_prod =
  nd->dot($X, $X->T);
```

Como

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 5 & 4 \\ 6 & 5 \end{bmatrix}$$

entonces

$$X^T = \begin{bmatrix} 1 & 2 & 5 & 6 \\ 1 & 2 & 4 & 5 \end{bmatrix}$$

La multiplicación matricial produce

$$X X^T = \begin{bmatrix} 2 & 4 & 9 & 11 \\ 4 & 8 & 18 & 22 \\ 9 & 18 & 41 & 50 \\ 11 & 22 & 50 & 61 \end{bmatrix}$$

Por ejemplo,

el elemento de la fila **C** y la columna **D** es

$$5 \times 6 + 4 \times 5 = 30 + 20 = 50$$

Ese valor representa

$$\mathbf{x}_C^T \mathbf{x}_D$$

La matriz completa generada por la operación `nd->dot(X, X->T)` contiene **todos los productos punto entre todos los pares de observaciones**.

Ya disponemos de los tres ingredientes necesarios para construir simultáneamente todas las distancias al cuadrado.

En la siguiente sección veremos cómo combinar

$$X_{\text{sum_squares}}$$

y

$$XX^T$$

para obtener la matriz completa de distancias sin calcular una sola distancia individualmente.

Construcción de la matriz `sqdistances`

En la diapositiva anterior obtuvimos dos resultados importantes.

Suma de cuadrados de cada observación

$$X_{\text{sum_squares}} = \begin{bmatrix} 2 \\ 8 \\ 41 \\ 61 \end{bmatrix}$$

Productos punto

$$XX^T = \begin{bmatrix} 2 & 4 & 9 & 11 \\ 4 & 8 & 18 & 22 \\ 9 & 18 & 41 & 50 \\ 11 & 22 & 50 & 61 \end{bmatrix}$$

La implementación en MXNet utiliza directamente la identidad

$$\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \mathbf{x}_i^T \mathbf{x}_i - 2\mathbf{x}_i^T \mathbf{x}_j + \mathbf{x}_j^T \mathbf{x}_j$$

mediante la instrucción

```
my $sqdistances =
    $X_sum_squares
    - (2 * $X_inner_prod)
    + $X_sum_squares->T;
```

Observemos las dimensiones.

$$X_{\text{sum_squares}} = \begin{bmatrix} 2 \\ 8 \\ 41 \\ 61 \end{bmatrix} \quad (4 \times 1)$$

Después de la transposición,

$$X_{\text{sum_squares}}^T = [2 \quad 8 \quad 41 \quad 61] \quad (1 \times 4)$$

Gracias al **broadcasting**, MXNet expande automáticamente ambos tensores hasta formar matrices de tamaño

$$4 \times 4$$

sin copiar explícitamente los datos.

Cálculo elemento por elemento

Veamos cómo se calcula cada elemento de la matriz.

Elemento (A,C)

Utilizamos la identidad

$$d^2 = \mathbf{x}_A^T \mathbf{x}_A - 2\mathbf{x}_A^T \mathbf{x}_C + \mathbf{x}_C^T \mathbf{x}_C$$

Sustituyendo los valores,

$$\begin{aligned} &= 2 - 2(9) + 41 \\ &= 25 \end{aligned}$$

Comprobemos el resultado utilizando la definición clásica.

$$(5 - 1)^2 + (4 - 1)^2 = 16 + 9 = 25$$

Los dos métodos producen exactamente el mismo resultado.

Elemento (B,D)

$$\begin{aligned} &8 - 2(22) + 61 \\ &= 25 \end{aligned}$$

Comprobación directa

$$(6 - 2)^2 + (5 - 2)^2 = 16 + 9 = 25$$

Nuevamente obtenemos el mismo resultado.

La ventaja es que MXNet calcula **todos los elementos simultáneamente**, sin utilizar ningún bucle explícito.

Matriz completa de distancias al cuadrado

Después del broadcasting y de las operaciones matriciales obtenemos

$$\begin{bmatrix} 0 & 2 & 25 & 41 \\ 2 & 0 & 13 & 25 \\ 25 & 13 & 0 & 2 \\ 41 & 25 & 2 & 0 \end{bmatrix}$$

Cada elemento representa

$$\|\mathbf{x}_i - \mathbf{x}_j\|^2$$

Observemos algunos ejemplos.

Par	Distancia al cuadrado
A-B	2
A-C	25
A-D	41
B-C	13
B-D	25
C-D	2

Estos valores coinciden exactamente con los obtenidos manualmente en la Parte 1B.1.

Antes de continuar, la implementación ejecuta

```
$sqdistances =  
  nd->clip(  
    $sqdistances,  
    a_min => 0.0,  
    a_max => 'Inf'  
  );
```

¿Por qué utilizar `clip()` ?

Debido a errores de redondeo en punto flotante, algunas distancias muy pequeñas pueden resultar ligeramente negativas.

Por ejemplo,

$$-2.4 \times 10^{-15}$$

En realidad esa distancia debería ser

$$0$$

`clip()` garantiza que todas las distancias sean mayores o iguales que cero, evitando problemas cuando posteriormente se calculen raíces cuadradas o probabilidades.

Construcción del vector condensado

La matriz de distancias al cuadrado contiene información duplicada.

$$D = D^T$$

Por ello solamente necesitamos conservar el triángulo superior.

La implementación utiliza

```
nd->triu_indices($n_samples,1)
```

Con cuatro observaciones se generan los índices

Fila	Columna
0	1
0	2
0	3
1	2
1	3
2	3

Posteriormente,

```
nd->gather_nd(
    $sqdistances,
    nd->triu_indices($n_samples,1)
)
```

extrae esos elementos de forma completamente vectorizada.

El resultado es

2	25	41	13	25	2
---	----	----	----	----	---

Este es exactamente el **vector condensado de distancias al cuadrado**.

Si posteriormente calculamos la raíz cuadrada de cada elemento obtenemos

1.4142	5.0000	6.4031	3.6056	5.0000	1.4142
--------	--------	--------	--------	--------	--------

que coincide exactamente con el cálculo manual realizado anteriormente.

Finalizamos esta parte indicando el código tensorial de la función pdist():

```
sub pdist {
    my ($self, $X) = @_;

    $X = $X->astype('float64');
    my $n_samples = $X->shape->[0];

    # ||x_i - x_j||^2 = x_i^2 - 2*x_i*x_j^T + x_j^2 (Matriz completa
    intermedia)
    my $X_sum_squares = nd->sum($X * $X, axis => 1, keepdims => 1);
    my $X_inner_prod = nd->dot($X, $X->T);
    my $sqdistances = $X_sum_squares - (2 * $X_inner_prod) +
    $X_sum_squares->T;
    $sqdistances = nd->clip($sqdistances, a_min => 0.0, a_max =>
    'Inf');

    # 1. Obtener los índices del triángulo superior estricto (offset k=1
    para omitir diagonal)
    # 2. Devuelve condensed al recolectar de forma puramente vectorial
    usando los índices estructurados
    return nd->gather_nd($sqdistances, nd->triu_indices($n_samples, 1));
}
```

Conclusión

La implementación de `pdist()` en MXNet:

1. Calcula todas las normas cuadradas.
2. Calcula todos los productos punto.
3. Construye la matriz completa de distancias al cuadrado mediante broadcasting.
4. Corrige pequeños errores numéricos con `clip()`.
5. Extrae el triángulo superior mediante `triu_indices()` y `gather_nd()`.
6. Devuelve el vector condensado sin utilizar un solo bucle explícito.

Este mismo vector será la entrada de la siguiente etapa del algoritmo t-SNE: el cálculo de las probabilidades condicionales mediante `_binary_search_perplexity`.

In []: